Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Catedrático: Marco Tulio Valdez Albizurez

Curso: Programación III

Tema del trabajo:

Árboles AVL

Nombre:	Carnet:
José Miguel Castillo Pérez	0907-24-1862

¿Qué es un árbol AVL?

Estos son estructuras de datos los cuales son una variante de los árboles binarios de búsqueda cuya gran distinción es que agregan un mecanismo de auto equilibrio dinámico. Esto garantiza que las operaciones como las de búsqueda, inserción y eliminación mantengan un rendimiento optimo incluso en los peores casos.

Estructura de un Árbol AVL

Estos al ser una variante de los arboles binarios de búsqueda estos al igual que los BST están compuestos por nodos y cada nodo tiene cierto valor además de poder tener como máximo 2 hijos uno a la izquierda y otro a la derecha.

Factor de balance

Este es el mecanismo de auto equilibrio mencionado anteriormente, para cualquier nodo el factor de balance se calcula así:

$$FB = Altura(Izquierda) - Altura(Derecha)$$

Un factor de balance 0 indica que los subárboles tienen la misma altura mientras que si es 1 significa que el subárbol izquierdo es más alto y por último que tenga un valor -1 significa que el subárbol derecho es el más alto.

Inserción en Árboles AVL

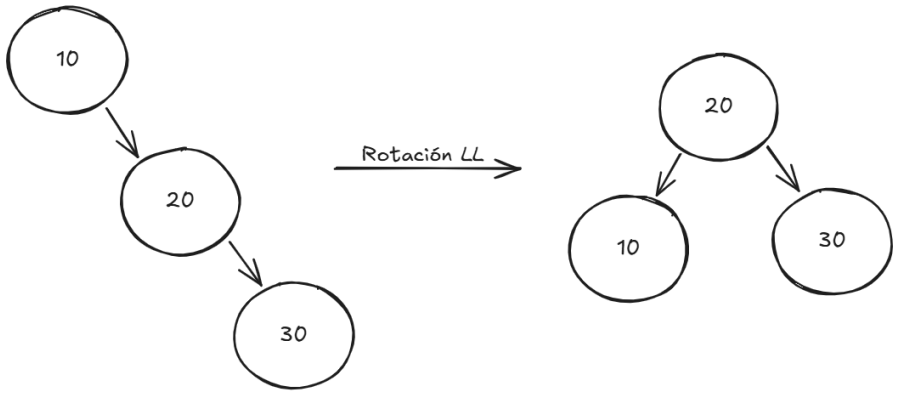
La inserción en este tipo de arboles se maneja de la misma forma que se haría en un árbol binario de búsqueda, pero incluye un paso adicional para verificar y corregir el factor de balance en resumen los pasos son los siguientes:

- Insertar el nuevo nodo en la posición adecuada, siguiendo las reglas de un BST.
- Actualizar las alturas de los nodos afectados en el camino desde el nodo insertado hasta la raíz.
- Calcular el factor de balance de cada nodo en este camino.
- Si el factor de balance de algún nodo no está en [-1, 0, 1], realizar las rotaciones necesarias para restaurar el equilibrio.
  (El Blog del Programador, 2025)

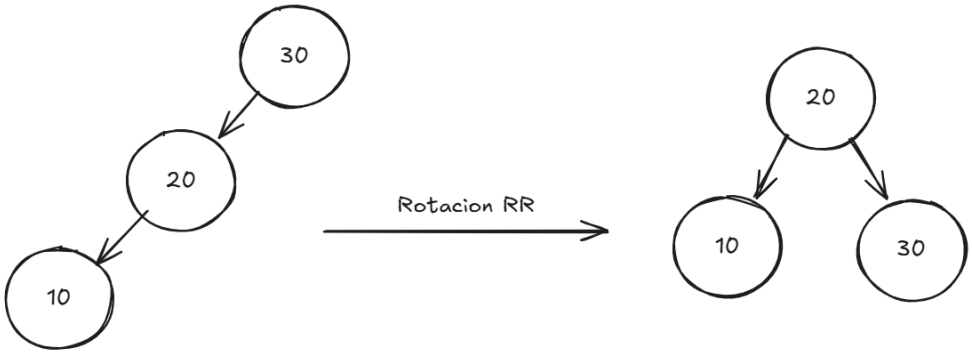
Rotaciones en Árboles AVL

Son el mecanismo principal para mantener el equilibrio en un árbol AVL, cada vez que una operación de inserción o eliminación causa un desbalance se aplica una o mas rotaciones para restaurar el balance.

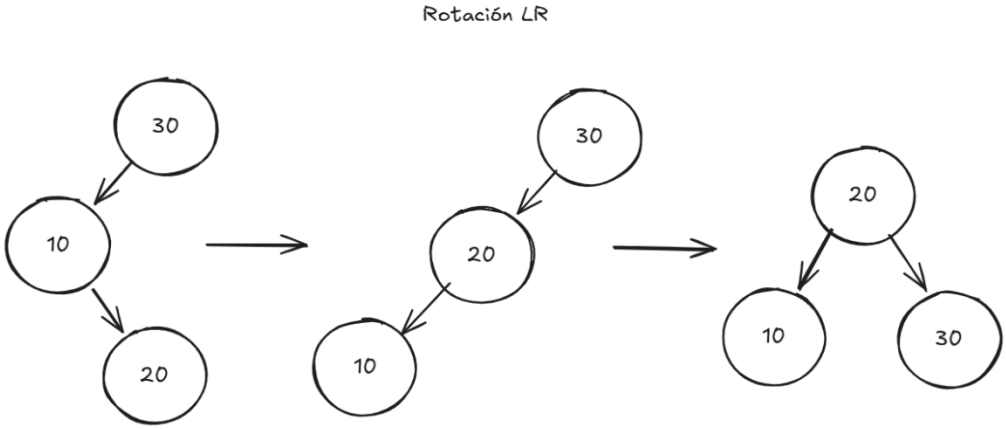
- **Rotación Izquierda (LL Rotation)**
  Se utiliza para evitar el desbalance en el subárbol derecho del hijo derecho de un nodo.



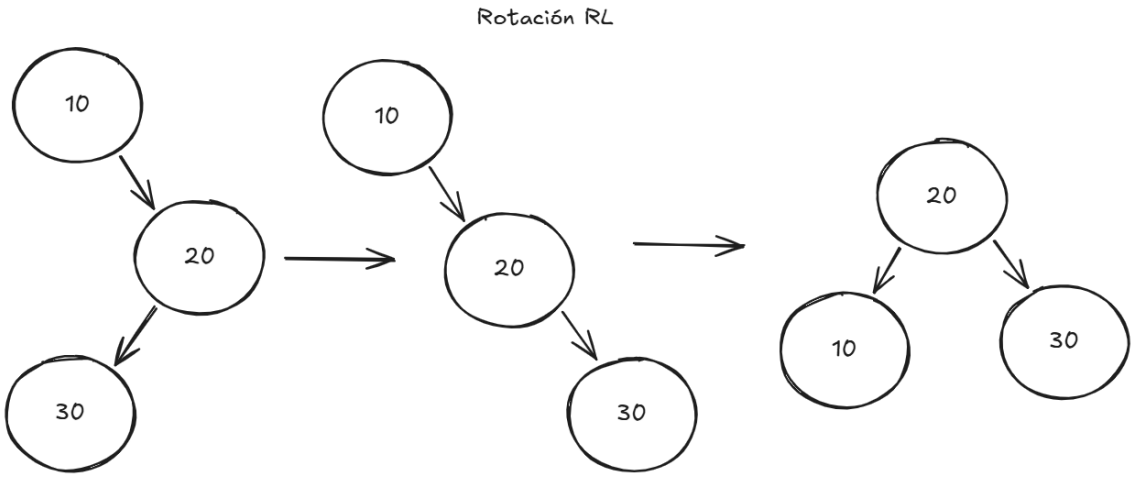
- **Rotación Derecha (RR Rotation)**
  Este es el opuesto de la rotación izquierda y se aplica cuando hay un desbalance en el subárbol izquierdo del hijo izquierdo.



- **Rotación Izquierda-Derecha (LR Rotation)**
  Esta es una combinación de una rotación izquierda seguida de una rotación derecha.



- **Rotación Derecha-Izquierda (RL Rotation)**
  Es el inverso de la anterior, una rotación derecha seguida de una izquierda.



Eliminación en Árboles AVL

Al igual que la inserción este proceso es realmente parecido al proceso de un árbol binario de búsqueda justamente teniendo como agregado un factor de balance al eliminar un nodo de la misma manera que la inserción lo hace y estos son los pasos para hacerlo:

- Localizar y eliminar el nodo según las reglas de un BST.
- Actualizar las alturas de los nodos en el camino desde el punto de eliminación hasta la raíz.
- Verificar el factor de balance de cada nodo en este camino.
- Realizar las rotaciones necesarias si se detecta un desbalance.
  (El Blog del Programador, 2025)

Complejidad de los Árboles AVL

Este tipo de árboles poseen una complejidad de $O(\log n)$ para las operaciones mencionadas anteriormente.

Link de Excalidraw.

Referencias

El Blog del Programador. (6 de 11 de 2025). *El Blog del Programador*. Obtenido de El Blog del Programador: https://elblogdelprogramador.com/posts/guia-completa-arboles-avl-programacion/#gsc.tab=0